Notebook Competitiva

ChichuMerge

2025

Índice

1. Template **2**

2. Comandos compilacion **2**

3. Divisores **2**

4. Tree Diameter **3**

5. Binary Lifting **3**

6. Articulation points **3**

7. Bridges **4**

8. Modular Integer **4**

9. GCD 128 bits iterativo **5**

10.Punto 3D **5**

11.Fast Pollard Rho **6**

12.FFT **7**

13.DP Bits Clasica **8**

14.DP Bits TSP **8**

15.DP Digitos **9**

16.Prefix Sum 2D **10**

17.Sliding Window Minimum 2D **10**

1. Template

```
#include <bits/stdc++.h>
#define forr(i, a, n) for(int i = a; i < n; i++)
#define forn(i, n) forr(i,0,n)
#define dfor(i, n) for(int i = n - 1; i >= 0; i--)
#define sz(a) ((int)a.size())
#define all(x) (x).begin(),(x).end()
#define dbg(x) cout << #x << " = " << (x) << endl
#define vdbg(x) {cout << '['; for(auto i : x) cout << i << ", "; cout << "]\n";}
#define ws <<' '<<
#define nl cout << '\n';

using namespace std;

typedef long long ll;
typedef pair<int, int> ii;

int main(){
  ios::sync_with_stdio(0);
  cin.tie(0);
  #ifdef chichu
    freopen("input.in", "r", stdin);
    //freopen("output.out", "w", stdout);
  #endif
  return 0;
}
```

2. Comandos compilacion

Uso	**Comando**
Compilar (debug)	`g++ -g -o nombreEjecutable archivo.cpp`
Debugger (gdb)	`gdb -q nombreEjecutable → run`
Otro debugger	`-g -fsanitize=address -fsanitize=undefined -D_GLIBCXX_DEBUG`
Compilación Geany	`g++ -Dflag -std=c++ -Wshadow -Wall -o "%e%fO2 -Wno-unused-result`
Ejecutar Geany	`"./%e"`

3. Divisores

Cuadro 1: Número con Máxima Cantidad de Divisores ($d(n)$) hasta 10^k

Límite (10^k)	**Número** (n)	**Divisores** ($d(n)$)
$k = 1$	6	4
$k = 2$	60	12
$k = 3$	840	32
$k = 4$	7 560	64
$k = 5$	83 160	128
$k = 6$	720 720	240
$k = 7$	8 648 640	448
$k = 8$	96 576 000	768
$k = 9$	735 134 400	1344
$k = 10$	9 657 600 000	2304
$k = 11$	81 939 600 000	4032
$k = 12$	901 335 600 000	6720
$k = 13$	9 914 691 600 000	10752
$k = 14$	98 138 438 400 000	17280
$k = 15$	883 245 945 600 000	26880
$k = 16$	9 715 705 392 000 000	41472
$k = 17$	98 138 438 400 000 000	64512

Cuadro 1 – *Continuación*

Límite (10^k)	**Número** (n)	**Divisores** ($d(n)$)
$k = 18$	932 315 164 800 000 000	103680

4. Tree Diameter

```
int ma = 0;
function<int(int, int)> dfs = [&](int s, int f) -> int {
    int ma1 = 0, ma2 = 0, val;

    for(int u : g[s]){ //saco los dos maximos caminos desde el nodo donde estoy y los sumo
        if(u == f) continue;
        val = dfs(u, s);
        if(val > ma1) {
            ma2 = ma1;
            ma1 = val;
        }
        else if(val > ma2) ma2 = val;
    }
    ma = max(ma, ma1 + ma2);
    return ma1 + 1;
};

dfs(0, -1);
```

5. Binary Lifting

```
int k = 30; //2^k es la maxima distancia que puede tener un salto
vector<vector<int>> sp(k, vector<int>(n)); //Cuidado: Nodos indexados en 0
forn(i, n) {
    cin >> x;
    sp[0][i] = x - 1; //Es necesario definir los saltos de tam 1 (aristas originales)
}

forr(i, 1, k){
    for(int j = 0; j < n; j++){
        sp[i][j] = sp[i - 1][sp[i - 1][j]]; //Mi salto actual es dos saltos de la pot anterior
    }
}

function<void(int&, int)> query = [&](int &x, int d){ //hago un salto de tam d desde x
    while(d){
        x = sp[__builtin_ctz(d)][x];
        d &= (d - 1); //apago el ultimo bit
    }
};

forn(i, q){
    cin >> x >> d;
    query(--x, d);
    cout << x + 1 << '\n';
}
```

6. Articulation points

```
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children=0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
```

```
            if (low[to] >= tin[v] && p!=-1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if(p == -1 && children > 1)
        IS_CUTPOINT(v);
}

void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs (i);
    }
}
```

7. Bridges

```
void IS_BRIDGE(int v,int to); // some function to process the found bridge
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    bool parent_skipped = false;
    for (int to : adj[v]) {
        if (to == p && !parent_skipped) {
            parent_skipped = true;
            continue;
        }
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v])
                IS_BRIDGE(v, to);
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}
```

8. Modular Integer

```
template <int MOD>
struct Mint {
private:
  unsigned int val;

public:
  Mint(){val=0;}
  template <typename T>
  constexpr Mint(T x) noexcept : val(x < 0      ? x % MOD + MOD
                                   : x >= MOD ? x % MOD
                                              : x) {}

  constexpr Mint &operator+() noexcept { return *this; }
  constexpr Mint &operator-() noexcept { return Mint(0) - *this; }

  constexpr Mint &operator+=(const Mint &rhs) noexcept {
```

```
    if ((val += rhs.val) >= MOD) val -= MOD;
    return *this;
  }
  constexpr Mint &operator-=(const Mint &rhs) noexcept {
    if ((val -= rhs.val) >= MOD) val += MOD;
    return *this;
  }
  constexpr Mint &operator*=(const Mint &rhs) noexcept {
    val = static_cast<unsigned long long>(val) * rhs.val % MOD;
    return *this;
  }

  constexpr Mint operator+(const Mint &rhs) const noexcept { return Mint(*this) += rhs; }
  constexpr Mint operator-(const Mint &rhs) const noexcept { return Mint(*this) -= rhs; }
  constexpr Mint operator*(const Mint &rhs) const noexcept { return Mint(*this) *= rhs; }

  constexpr Mint &operator/=(const Mint &rhs) noexcept { return *this *= rhs.inv(); }
  constexpr Mint operator/(const Mint &rhs) const noexcept { return Mint(*this) /= rhs; }

  constexpr Mint pow(long long n) const noexcept {
    Mint ret(1), mul(val);
    while (n) {
      if (n % 2) ret *= mul;
      mul *= mul;
      n >>= 1;
    }
    return ret;
  }

  constexpr Mint inv() const noexcept { return pow(MOD - 2); }

  constexpr bool operator==(const Mint &rhs) const noexcept { return val == rhs.val; }
  constexpr bool operator!=(const Mint &rhs) const noexcept { return val != rhs.val; }

  friend constexpr Mint operator+(long long lhs, const Mint &rhs) { return Mint(lhs) += rhs; }
  friend constexpr Mint operator-(long long lhs, const Mint &rhs) { return Mint(lhs) -= rhs; }
  friend constexpr Mint operator*(long long lhs, const Mint &rhs) { return Mint(lhs) *= rhs; }
  friend constexpr Mint operator/(long long lhs, const Mint &rhs) { return Mint(lhs) /= rhs; }

  friend constexpr bool operator==(long long lhs, const Mint &rhs) { return lhs == rhs.val; }
  friend constexpr bool operator!=(long long lhs, const Mint &rhs) { return lhs != rhs.val; }

  friend std::ostream &operator<<(std::ostream &os, const Mint<MOD> &x) { return os << x.val; }
  friend std::istream &operator>>(std::istream &is, Mint<MOD> &x) {
    long long val_;
    is >> val_;
    x = val_;
    return is;
  }
};
constexpr int mod = 998244353;
using mint = Mint<mod>;
```

9. GCD 128 bits iterativo

```
 __int128_t gcd(__int128_t a, __int128_t b) {
    if (!a || !b) return a | b;
    unsigned shift = __builtin_ctz(a | b);
    a >>= __builtin_ctz(a);
    do {  b >>= __builtin_ctz(b);
      if (a > b)
        swap(a, b);
      b -= a;
    } while (b);
    return a << shift;
}
```

10. Punto 3D

```
typedef long double T;
const T INF = 3e18;
#define op operator
const T eps=1e-9;
struct Pto{
    T x,y,z;
    Pto(): x(0),y(0),z(0) {}
    Pto(T a, T b, T c): x(a),y(b),z(c) {}
    //Pto(pto a) : x(a.x), y(a.y), z(0) {}
    Pto op+(const Pto &p){return{x+p.x,y+p.y,z+p.z};}
```

```
    Pto op-(const Pto &p){return{x-p.x,y-p.y,z-p.z};}
    Pto op*(const T d){return{x*d,y*d,z*d};}
    Pto op/(const T d){return{x/d,y/d,z/d};}
    bool op==(const Pto &p){return fabsl(x-p.x)<=eps&&fabsl(y-p.y)<=eps&&fabsl(z-p.z)<=eps;}
    T operator|(Pto v){return v.x*x+v.y*y+v.z*z;}
    T sq(){return (*this)|(*this);}
    ld abs(){return sqrtl((ld)(*this).sq());}
    Pto unit(){return *this/abs();}
    Pto operator*(Pto w){return {y*w.z - z*w.y, z*w.x - x*w.z, x*w.y - y*w.x};}
};
ld angle(Pto v, Pto w){
    ld cosTheta=(v|w)/(v.abs()*w.abs());
    return acosl(max((ld)-1.0,min((ld)1.0,cosTheta)));
}
// >0 => s mismo lado que n a plano pqr
T orient(Pto p, Pto q, Pto r, Pto s){return (q-p)*(r-p)|(s-p);}
```

11. Fast Pollard Rho

```
#include<bits/stdc++.h>
using namespace std;

using ll = long long;
namespace PollardRho {
  mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
  const int P = 1e6 + 9;
  ll seq[P];
  int primes[P], spf[P];
  inline ll add_mod(ll x, ll y, ll m) {
    return (x += y) < m ? x : x - m;
  }
  inline ll mul_mod(ll x, ll y, ll m) {
    ll res = __int128(x) * y % m;
    return res;
    // ll res = x * y - (ll)((long double)x * y / m + 0.5) * m;
    // return res < 0 ? res + m : res;
  }
  inline ll pow_mod(ll x, ll n, ll m) {
    ll res = 1 % m;
    for (; n; n >>= 1) {
      if (n & 1) res = mul_mod(res, x, m);
      x = mul_mod(x, x, m);
    }
    return res;
  }
  // O(it * (logn)^3), it = number of rounds performed
  inline bool miller_rabin(ll n) {
    if (n <= 2 || (n & 1 ^ 1)) return (n == 2);
    if (n < P) return spf[n] == n;
    ll c, d, s = 0, r = n - 1;
    for (; !(r & 1); r >>= 1, s++) {}
    // each iteration is a round
    for (int i = 0; primes[i] < n && primes[i] < 32; i++) {
      c = pow_mod(primes[i], r, n);
      for (int j = 0; j < s; j++) {
        d = mul_mod(c, c, n);
        if (d == 1 && c != 1 && c != (n - 1)) return false;
        c = d;
      }
      if (c != 1) return false;
    }
    return true;
  }
  void init() {
    int cnt = 0;
    for (int i = 2; i < P; i++) {
      if (!spf[i]) primes[cnt++] = spf[i] = i;
      for (int j = 0, k; (k = i * primes[j]) < P; j++) {
        spf[k] = primes[j];
        if (spf[i] == spf[k]) break;
      }
    }
  }
  // returns O(n^(1/4))
  ll pollard_rho(ll n) {
    while (1) {
      ll x = rnd() % n, y = x, c = rnd() % n, u = 1, v, t = 0;
      ll *px = seq, *py = seq;
      while (1) {
        *py++ = y = add_mod(mul_mod(y, y, n), c, n);
        *py++ = y = add_mod(mul_mod(y, y, n), c, n);
        if ((x = *px++) == y) break;
        v = u;
```

```
        u = mul_mod(u, abs(y - x), n);
        if (!u) return __gcd(v, n);
        if (++t == 32) {
          t = 0;
          if ((u = __gcd(u, n)) > 1 && u < n) return u;
        }
      }
      if (t && (u = __gcd(u, n)) > 1 && u < n) return u;
    }
  }
  vector<ll> factorize(ll n) {
    if (n == 1) return vector <ll>();
    if (miller_rabin(n)) return vector<ll> {n};
    vector <ll> v, w;
    while (n > 1 && n < P) {
      v.push_back(spf[n]);
      n /= spf[n];
    }
    if (n >= P) {
      ll x = pollard_rho(n);
      v = factorize(x);
      w = factorize(n / x);
      v.insert(v.end(), w.begin(), w.end());
    }
    return v;
  }
}
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  PollardRho::init();
  int t; cin >> t;
  while (t--) {
    ll n; cin >> n;
    auto f = PollardRho::factorize(n);
    sort(f.begin(), f.end());
    cout << f.size() << ' ';
    for (auto x: f) cout << x << ' '; cout << '\n';
  }
  return 0;
}
```

12. FFT

```
#include <bits/stdc++.h>
using namespace std;
#define forsn(i,s,n) for(int i = int(s);i<int(n);i++)
#define dforsn(i,s,n) for(int i = int(n)-1;i>=int(s);i--)
#define fore(i,s,n) forsn(i,s,n)
#define forn(i,n) forsn(i,0,n)
#define dforn(i,n) dforsn(i,0,n)
#define sz(a) ((int)a.size())

// The maximum length of the resulting
// convolution vector is 2^LG
const int LG = 23;
using ll = long long;

template<class u, class uu, u p, u root>
struct FFT {
    u r[1+(2<<LG)];
    constexpr u m(u a, u b) {
        uu k = uu(a)*b;
        #define op(g) g*(g*p+2)
        k += u(k) * (op(op(op(op(op(-p)))))) * uu(p);
        #undef op
        return u(k>>(8*sizeof(u)));
    }
    constexpr u red(u k, u a) { return a-k*(a>=k); }
    FFT() {
        u k = r[2<<LG] = -p%p, b=root, e = p>>LG;
        for(; e; e/=2, b=m(b,b)) if(e%2) k=m(k, b);
        dforn(i, 2<<LG) r[i]=red(p, m(r[i+1], k)), i&(i-1)?0:k=m(k,k);
        assert(r[2] != r[3]); assert(r[1] == r[2]);
    }
    vector<ll> cv(const vector<ll> &as, const vector<ll> &bs, u *v) {
        int c=max(sz(as)+sz(bs)-1, 0), n=1;
        assert(c <= (1<<LG));
        u h=u(uu(-p)*-p%p), a=m(h, p/2+1), x, y;
        while(n<c) n*=2, h=red(p, m(h, a));
        forn(i, n)
        v[i] = i<sz(as) ? u(as[i]) : 0,
        v[i+n] = i<sz(bs) ? u(bs[i]) : 0;
```

```
        for(auto s:{v,v+n})
        dforsn(j, 2, n+1) for(int k=j&-j; k/=2;) forsn(i, j-k, j)
        x=s[i], y=s[i-k],
        s[i-k] = red(2*p, x+y),
        s[i] = m(2*p+y-x, r[3*k-j+i]);
        forn(i, n) v[i] = m(v[i], v[i+n]);
        forsn(j, 2, n+1) for(int k=1; !(k&j); k*=2) forsn(i, j-k, j)
        x = m(v[i], r[3*k+j-i]),
        y = red(2*p, v[i-k]),
        v[i-k]=x+y, v[i]=2*p+y-x;
        forn(i, c) v[i] = red(p, m(v[i], h));
        return vector<ll>(v, v+c);
    }
    };

    // For modular convolutions modulo 998244353:
    vector<ll> conv_small(const vector<ll> &as, const vector<ll> &bs) {
    static uint32_t v[2<<LG];
    static FFT<uint32_t, uint64_t, 998244353, 3> fft; //tambien puede ser 469762049
    return fft.cv(as, bs, v);
    }

    // For modular convolutions modulo a 62 bit prime:
    vector<ll> conv_big(const vector<ll> &as, const vector<ll> &bs) {
    static uint64_t v[2<<LG];
    static FFT<uint64_t, __uint128_t, (1ull<<62)-(18ull<<32)+1, 3> fft;
    return fft.cv(as, bs, v);
    }

    // For modular convolutions modulo an arbitrary 32-bit modulus:
    vector<ll> conv_sunzi(const vector<ll> &v1, const vector<ll> &v2, ll m) {
    const uint64_t inv = 2703402103339935109ull,
        mod1 = (1ull<<62)-(18ull<<32)+1,
        mod2 = (1ull<<62)-(76ull<<32)+1;
    static_assert(__uint128_t(inv)*mod2%mod1 == (-mod1)%mod1);
    static uint64_t v[2<<LG];
    static FFT<uint64_t, __uint128_t, mod1, 3> fft1;
    static FFT<uint64_t, __uint128_t, mod2, 17> fft2;
    auto as=fft1.cv(v1, v2, v), bs=fft2.cv(v1, v2, v);
    forn(i, sz(as)) {
        auto d = fft1.m(mod1+as[i]-bs[i], inv);
        d -= mod1*(d >= mod1); d %= m;
        as[i] = (bs[i] + mod2%m*d)%m;
    }
    return as;
}
```

13. DP Bits Clasica

```
    int n, x; //Permite reducir permutaciones a subconjuntos n! -> 2^n
    cin >> n >> x;
    vector<int> v(n);
    forn(i, n) cin >> v[i];
    ii best[1 << n];
    best[0] = {1, 0}; //En este caso llevo un par pero podria llevar un solo elemento

    forr(s, 1, (1 << n)){
        best[s] = {n + 1, 0};
        forn(p, n){
            if(s & (1 << p)){
                ii option = best[s ^ (1 << p)];
                if(option.sc + v[p] <= x){
                    option.sc += v[p];
                }
                else{
                    option.fr++;
                    option.second = v[p];
                }
                best[s] = min(best[s], option);
            }
        }
    }

    cout << best[(1 << n) - 1].fr << '\n';
```

14. DP Bits TSP

```
int tsp(int u, int mask){
    if(mask == 0) return 0;
```

```
    int &ans = memo[u][mask];
    if(ans != -1) return ans;
    ans = inf;
    int m = mask;
    while(m){
        int pot2x = ( m & (-m) );
        int x = __builtin_ctz(pot2x)+1;
        ans = min(ans, dist[u][x] + tsp(u, mask - pot2x));
        m -= pot2x;
    }
    return ans;
}
```

15. DP Digitos

```
///How many numbers x are there in the range a to b, where the digit d occurs exactly k times in x?

#include <bits/stdc++.h>
using namespace std;

vector<int> num;
int a, b, d, k;
int DP[12][12][2];
/// DP[p][c][f] = Number of valid numbers <= b from this state
/// p = posicion actual de 0 a n
/// c = cantidad de veces que aparece el digito (podria ser otra cosa)
/// f = bandera para saber si somos menores a b o no

int call(int pos, int cnt, int f){
    if(cnt > k) return 0;

    if(pos == num.size()){
        if(cnt == k) return 1;
        return 0;
    }

    if(DP[pos][cnt][f] != -1) return DP[pos][cnt][f];
    int res = 0;

    int LMT;

    if(f == 0){
        //como el numero viene empatado con b el limite es el valor de b
        LMT = num[pos];
    } else {
        //como ya es menor podemos poner cualquier digito
        LMT = 9;
    }

    //Intentamos poner un digito sin excedernos de b
    for(int dgt = 0; dgt<=LMT; dgt++){
        int nf = f;
        int ncnt = cnt;
        if(f == 0 && dgt < LMT) nf = 1; //el numero ya es menor a b
        if(dgt == d) ncnt++;
        if(ncnt <= k) res += call(pos+1, ncnt, nf);
    }

    return DP[pos][cnt][f] = res;
}

int solve(int b){
    num.clear();
    while(b>0){
        num.push_back(b%10);
        b/=10;
    }
    reverse(num.begin(), num.end()); //Asi es mas practico de almacenar

    memset(DP, -1, sizeof(DP));
    int res = call(0, 0, 0);
    return res;
}

int main() {
    //Recordar que a veces es importante tener una bandera para manejar los 0000 a la izquierda. La
        bandera nos dice si ya empezamos a construir el numero o todavia no
    //Si es ll tendremos que agrandar la cantidad de posiciones, son 19 digitos
    //La complejidad es la multiplicacion de todos los estados
    cin >> a >> b >> d >> k;
    int res = solve(b) - solve(a-1);
    cout << res << '\n';

```

```
    return 0;
}
```

16. Prefix Sum 2D

```
vector<vector<int>> ps;
vector<vector<int>> v;
int n;
void cargarPrefixSum(){
    forr(i, 1, n + 1){
        forr(j, 1, n + 1){
            ps[i][j] = v[i - 1][j - 1] + ps[i - 1][j] + ps[i][j - 1] - ps[i - 1][j - 1];
        }
    }
}

int query(int x1, int y1, int x2, int y2){
    int ans = ps[x2][y2] - ps[x1 - 1][y2] - ps[x2][y1 - 1] + ps[x1 - 1][y1 - 1];
    return ans;
}

//ps.resize; v.resize; cargarPrefixSum();
//No hace falta modificar las queries salvo que vengan indexadas en 0 (hacer ++). ll ?
```

17. Sliding Window Minimum 2D

```
int n, m, r, c; cin >> n >> m;
vector<vector<int>> v(n, vector<int>(m));
forn(i, n) forn(j, m) cin >> v[i][j];
cin >> r >> c;

forn(i, n){
    deque<ii> q;
    dfor(j, m) {
        while(sz(q) and q.back().first <= v[i][j]) q.pop_back();
        if(sz(q) and q.front().second >= j + c) q.pop_front();
        q.push_back({v[i][j], j});
        v[i][j] = q.front().first;
    }
}

forn(j, m - c + 1){
    deque<ii> q;
    dfor(i, n) {
        while(sz(q) and q.back().first <= v[i][j]) q.pop_back();
        if(sz(q) and q.front().second >= i + r) q.pop_front();
        q.push_back({v[i][j], i});
        v[i][j] = q.front().first;
    }
}

forn(i, n - r + 1) forn(j, m - c + 1) cout << v[i][j] << (j == m - c ? '\n' : ' ');
```